

CURSO REACT Y NODE.JS - CLASE 2 DE 6

Stack básico, rutas internas y página de contacto

Documento ampliado para explicar el segundo paso del curso: dependencias, arquitectura, código completo, flujo de React Router y sintaxis JavaScript/JSX usada en este stack.

Duración

5 horas

Nivel

Principiante

Producto

Inicio + Contacto + Backend
preparado

Índice

- | | |
|--|------------------------------------|
| 1. Objetivos de la clase | 2. Orden de trabajo |
| 3. Qué vamos a construir | 4. Dependencias y comandos |
| 5. Arquitectura del stack | 6. Flujo completo de la app |
| 7. Código archivo por archivo | 8. Backend preparado |
| 9. JavaScript y JSX usados en este stack | 10. Ejercicios de JavaScript y JSX |
| 11. Ejercicios de stack | |

1. Objetivos de la clase

En la primera clase vimos cómo preparar el entorno y crear un primer Hola Mundo con React y Vite. En esta segunda clase convertimos ese ejemplo inicial en una estructura mínima de aplicación. Todavía no buscamos una app completa, sino una base clara para crecer.

El cambio principal respecto a la clase 1 es conceptual. Ya no pensamos en una única pantalla dentro de `App.jsx`. Empezamos a pensar en una aplicación formada por piezas: una página de inicio, una página de contacto, una navegación común y un archivo donde se declaran las rutas. Esa separación es pequeña, pero prepara al alumnado para proyectos más reales.

AL TERMINAR

- Entenderemos que una app React puede tener varias URLs internas.
- Instalaremos la dependencia adicional `react-router-dom`.
- Separaremos el código en `components/`, `pages/` y `routes/`.
- Crearemos una ruta de inicio `/` y una ruta de contacto `/contacto`.
- Usaremos `Link` para navegar sin recargar toda la página.
- Dejaremos una carpeta `Backend/` preparada, sin desarrollar Express todavía.
- Relacionaremos cada archivo con el flujo real de la aplicación.
- Repasaremos la sintaxis JavaScript y JSX que aparece en el código.

ALCANCE DE CLASE 2

No vamos a crear todavía formularios funcionales, estado con `useState`, peticiones HTTP, Express ni bases de datos. Esta clase trata de estructura, navegación y lectura del flujo.

2. Orden de trabajo

Para evitar líos, la clase 2 debe seguir siempre el mismo orden. No empezamos creando rutas si antes no hemos instalado la dependencia, y no creamos enlaces si antes no tenemos claras las páginas.

Este orden también ayuda a explicar causa y efecto. Primero creamos las páginas porque son el contenido que queremos mostrar. Después creamos las rutas porque necesitamos decir qué URL muestra cada página. Luego creamos la navegación porque los enlaces necesitan apuntar a rutas que ya existen. Por último, conectamos todo en `App.jsx`.

SECUENCIA RECOMENDADA

1. Partir del proyecto React creado en la clase 1.
2. Abrir la raíz del proyecto en VS Code.
3. Comprobar que estamos en la carpeta correcta desde la terminal.
4. Instalar dependencias si el proyecto acaba de descargarse.
5. Instalar la dependencia nueva de rutas: `react-router-dom`.
6. Crear las carpetas `components/`, `pages/` y `routes/`.
7. Crear componentes pequeños: `Texto.jsx` y `Boton.jsx`.
8. Crear después las páginas: `Inicio.jsx` y `Contacto.jsx`.
9. Crear después `AppRoutes.jsx` para asociar URLs con páginas.
10. Crear `Navegacion.jsx` con enlaces internos mediante `Link`.
11. Actualizar `App.jsx` para montar `BrowserRouter`, navegación y rutas.
12. Preparar la carpeta `Backend/` sin desarrollar Express todavía.
13. Probar el frontend con `npm run dev`.
14. Verificar con `npm run lint` y `npm run build`.

2.1 Comandos en orden

Si el proyecto ya está creado y estamos empezando la clase 2, estos son los comandos principales:

```
npm install
npm install react-router-dom
npm run dev
```

Si queremos comprobar que todo compila al final:

```
npm run lint
npm run build
```

En clase conviene escribir los comandos en la terminal integrada de VS Code y detenerse después de cada uno. Si el comando falla, no se debe seguir al siguiente paso. Por ejemplo, si `npm install react-router-dom` falla, los imports de `BrowserRouter`, `Routes`, `Route` o `Link` no funcionarán.

2.2 Archivos en orden de creación

Orden	Archivo o carpeta	Motivo
1	<code>src/components/Texto.jsx</code>	Componente pequeño de texto para usar dentro de Inicio.
2	<code>src/components/Boton.jsx</code>	Componente visual de botón, sin eventos todavía.
3	<code>src/pages/Inicio.jsx</code>	Primera página: se mostrará en <code>/</code> .
4	<code>src/pages/Contacto.jsx</code>	Segunda página: se mostrará en <code>/contacto</code> .
5	<code>src/routes/AppRoutes.jsx</code>	Une cada URL con su página.
6	<code>src/components/Navegacion.jsx</code>	Permite navegar entre Inicio y Contacto.
7	<code>src/App.jsx</code>	Monta la estructura común: router, navegación y rutas.
8	<code>Backend/</code>	Reserva el lugar donde vivirá Node/Express más adelante.

EVITAR

No conviene crear todo dentro de `App.jsx`. Para clase 2 queremos enseñar desde el principio que una aplicación se separa por responsabilidades: páginas, componentes y rutas.

3. Qué vamos a construir

El objetivo práctico es tener un stack muy pequeño con dos pantallas: una página de inicio y una página de contacto. El backend solo queda preparado como carpeta para que se vea la arquitectura global.

Este stack no tiene todavía datos dinámicos, formularios, estado ni llamadas a una API. Eso es intencionado. La clase se centra en que el alumnado entienda qué archivo cumple cada función y cómo se conectan las piezas. Una vez entendido esto, será mucho más fácil añadir interactividad.

URL	Archivo de página	Qué muestra
<code>/</code>	<code>src/pages/Inicio.jsx</code>	Datos básicos del curso y bloque de conceptos.

URL	Archivo de página	Qué muestra
/contacto	src/pages/Contacto.jsx	Página interna de contacto con texto explicativo.

Para navegar entre esas páginas usamos un componente llamado `Navegacion`. Para decidir qué página se muestra según la URL usamos `AppRoutes`.

EJEMPLO MENTAL

Podemos imaginar `AppRoutes.jsx` como una tabla de equivalencias: "si la URL es esta, muestra esta página". La navegación no decide qué se muestra; solo cambia la URL. La decisión la toman las rutas.

4. Dependencias y comandos

Una dependencia es un paquete externo que nuestro proyecto necesita para funcionar. En esta clase debemos distinguir entre las dependencias que ya trae la plantilla de Vite con React y la dependencia nueva que instalamos para las rutas.

El archivo `package.json` es como una ficha técnica del proyecto. Indica qué comandos podemos ejecutar y qué paquetes necesita la aplicación. Cuando ejecutamos `npm install`, npm lee esa ficha y descarga los paquetes necesarios.

4.1 Dependencias base del proyecto React

Dependencia	Para qué sirve
<code>react</code>	Librería principal para construir componentes e interfaces.
<code>react-dom</code>	Permite montar React dentro del DOM del navegador.

4.2 Dependencia adicional de clase 2

Dependencia	Por qué la añadimos	Comando
<code>react-router-dom</code>	Permite crear rutas internas como <code>/</code> y <code>/contacto</code> .	<code>npm install react-router-dom</code>

Comando que hay que ejecutar en consola desde la raíz del proyecto:

```
npm install react-router-dom
```

Después de instalarlo, aparecerá en `package.json` dentro de `dependencies`:

```
"dependencies": {  
  "react": "^19.2.6",  
  "react-dom": "^19.2.6",  
  "react-router-dom": "^7.15.1"  
}
```

En esta clase, `react-router-dom` aporta cuatro piezas importantes:

Pieza	Dónde se usa	Para qué sirve
<code>BrowserRouter</code>	<code>App.jsx</code>	Activa el sistema de rutas en la aplicación.
<code>Routes</code>	<code>AppRoutes.jsx</code>	Agrupar todas las rutas disponibles.

Pieza	Dónde se usa	Para qué sirve
Route	AppRoutes.jsx	Relaciona una URL concreta con un componente.
Link	Navegacion.jsx	Crea enlaces internos sin recargar toda la página.

4.3 Dependencias frente a devDependencies

En `package.json` aparecen dos bloques habituales: `dependencies` y `devDependencies`. Para el alumnado principiante, la diferencia se puede explicar así:

- `dependencies`: paquetes que la aplicación necesita para funcionar.
- `devDependencies`: herramientas necesarias durante el desarrollo.

React y React Router son dependencias de la aplicación. Vite y ESLint son herramientas de desarrollo.

4.4 Comandos importantes del proyecto

Comando	Cuándo se usa	Qué hace
<code>npm install</code>	Al descargar o crear el proyecto.	Instala las dependencias en <code>node_modules</code> .
<code>npm run dev</code>	Durante el desarrollo.	Arranca Vite en <code>http://localhost:5173/</code> .
<code>npm run build</code>	Para comprobar que compila.	Genera una versión final en <code>dist/</code> .
<code>npm run lint</code>	Para revisar errores de código.	Ejecuta ESLint sobre el proyecto.

PARA CLASE

`dist/` se genera al ejecutar `npm run build`. No es necesario mantenerlo visible durante la explicación, porque es una carpeta generada automáticamente.

4.5 De dónde sale `eslint.config.js`

El archivo `eslint.config.js` lo crea la plantilla de Vite con React. No lo hemos escrito desde cero. Viene preparado para que el comando `npm run lint` pueda revisar nuestro código.

ESLint no arranca la aplicación y no crea la interfaz. Su papel es analizar los archivos `.js` y `.jsx` para detectar errores o patrones problemáticos. Por ejemplo, ayuda a encontrar imports mal usados, variables sin usar o errores relacionados con React.

5. Arquitectura del stack

La raíz del proyecto contiene el frontend con React y Vite. La carpeta `Backend/` queda reservada para Node y Express. Aunque estén en la misma carpeta de trabajo, frontend y backend son responsabilidades diferentes.

Una regla útil para decidir dónde colocar un archivo es preguntarse: "¿este archivo representa una página completa, una pieza reutilizable o una configuración de rutas?". Si es una pantalla completa, irá a `pages/`. Si es una pieza que puede aparecer en varias páginas, irá a `components/`. Si decide qué URL muestra qué página, irá a `routes/`.

```
00_PROYECTO_PROFE/  
  public/  
  src/  
    components/  
      Boton.jsx  
      Navegacion.jsx  
      Texto.jsx  
    pages/  
      Inicio.jsx  
      Contacto.jsx  
    routes/  
      AppRoutes.jsx  
  App.jsx  
  App.css  
  index.css  
  main.jsx  
Backend/  
  index.js  
  package.json  
  README.md  
index.html  
eslint.config.js  
package.json  
package-lock.json  
vite.config.js
```

Zona	Responsabilidad	Ejemplo
<code>index.html</code>	HTML inicial que carga el navegador.	Contiene <code>div#root</code> y carga <code>main.jsx</code> .
<code>src/main.jsx</code>	Punto de entrada de React.	Monta <code>App</code> dentro de <code>div#root</code> .
<code>src/App.jsx</code>	Estructura común de la app.	Contiene <code>BrowserRouter</code> , navegación y rutas.
<code>src/components/</code>	Componentes reutilizables.	<code>Navegacion.jsx</code> , <code>Texto.jsx</code> , <code>Boton.jsx</code> .
<code>src/pages/</code>	Páginas completas.	<code>Inicio.jsx</code> y <code>Contacto.jsx</code> .

Zona	Responsabilidad	Ejemplo
<code>src/routes/</code>	Declaración de URLs.	<code>AppRoutes.jsx</code> .
<code>eslint.config.js</code>	Configuración de revisión de código.	Se usa al ejecutar <code>npm run lint</code> .
<code>Backend/</code>	Servidor Node futuro.	<code>index.js</code> preparado sin Express todavía.

5.1 Ejemplos de decisión de carpeta

Necesidad	Carpeta	Motivo
Crear una página "Cursos".	<code>src/pages/</code>	Es una pantalla completa asociada a una URL.
Crear un menú reutilizable.	<code>src/components/</code>	Puede aparecer en varias páginas.
Añadir la URL <code>/cursos</code> .	<code>src/routes/</code>	Las rutas se declaran en un archivo específico.
Preparar el servidor Node.	<code>Backend/</code>	No forma parte del código React.

6. Flujo completo de la app

Este es el recorrido más importante de la clase. Conviene explicarlo despacio y volver a él varias veces.

La clave es distinguir entre "cargar la aplicación" y "navegar dentro de la aplicación". La primera carga ocurre cuando abrimos `localhost:5173`. A partir de ahí, React Router puede cambiar entre páginas internas sin volver a cargar todo desde cero.

1. Vite

Sirve `index.html` en el navegador.

2. index.html

Tiene `div#root` y carga `main.jsx`.

3. main.jsx

Busca `root` y renderiza `App`.

4. App.jsx

Activa `BrowserRouter`.

5. Navegacion.jsx

Muestra enlaces con `Link`.

6. AppRoutes.jsx

Relaciona URL con página.

6.1 Flujo al abrir la home

1. El navegador abre `http://localhost:5173/`.
2. Vite entrega `index.html`.
3. `index.html` carga `/src/main.jsx`.
4. `main.jsx` monta `App` en `div#root`.
5. `App` activa `BrowserRouter`.
6. `AppRoutes` ve que la URL es `/`.
7. React muestra `Inicio`.

6.2 Flujo al pulsar Contacto

1. El usuario pulsa el enlace `Contacto`.
2. `Link` cambia la URL a `/contacto`.
3. `BrowserRouter` informa a React Router de la ruta actual.
4. `AppRoutes` encuentra `path="/contacto"`.
5. React muestra el componente `Contacto`.

FRASE CLAVE

`App.jsx` no pregunta manualmente qué URL hay. `AppRoutes.jsx` declara las rutas y React Router decide qué página debe renderizar.

7. Código archivo por archivo

En esta sección aparece el código de los archivos implicados en el stack de clase 2. La idea es que el alumnado pueda seguir el flujo con el documento delante.

7.1 index.html

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Curso React y Node.js</title>
  </head>
  <body>
    <!-- Punto de montaje: React colocara aqui la aplicacion desde
src/main.jsx. -->
    <div id="root"></div>

    <!-- Punto de entrada: este archivo inicia React. -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Lo importante para el flujo no son los metadatos del HTML. Lo importante es `div#root` y el script que carga `src/main.jsx`.

7.2 vite.config.js

```
// defineConfig viene de Vite.
// Sirve para escribir la configuracion del proyecto con ayudas del editor.
import { defineConfig } from 'vite'

// react es el plugin oficial de Vite para proyectos React.
// Permite que Vite entienda JSX y actualice la pantalla mientras
desarrollamos.
import react from '@vitejs/plugin-react'

// Exportamos la configuracion que Vite usara al ejecutar npm run dev o npm run
build.
export default defineConfig({
  plugins: [react()],
})
```

7.3 eslint.config.js

```
// Este archivo lo crea la plantilla de Vite con React.
// Su funcion es configurar ESLint, la herramienta que revisa el codigo cuando
// ejecutamos npm run lint. ESLint no ejecuta la aplicacion: solo analiza
archivos
// .js y .jsx para detectar errores, malas practicas o configuraciones que no
// encajan con React, como problemas con hooks o con la recarga rapida de Vite.
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.js', '**/*.jsx'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
],
)
```

Este archivo aparece en la raíz porque afecta a todo el proyecto. En clase no necesitamos memorizar cada línea: basta con entender que configura la revisión de código que se ejecuta con `npm run lint`.

7.4 package.json

```
{
  "name": "curso-react-node",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "react-router-dom": "^7.15.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
```

```

    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "vite": "^8.0.12"
  }
}

```

En JSON no se escriben comentarios. Por eso explicamos este archivo en el documento, pero no lo comentamos dentro del propio `package.json`.

7.5 `src/main.jsx`

```

// Importamos StrictMode desde React.
// StrictMode ayuda durante el desarrollo avisando de posibles problemas.
// En este curso lo usamos solo aqui, en el punto de entrada de la aplicacion.
import { StrictMode } from 'react'

// Importamos createRoot desde react-dom/client.
// Sirve para decirle a React en que elemento del HTML debe montar la
// aplicacion.
// Tambien se usa una vez, en main.jsx.
import { createRoot } from 'react-dom/client'

// Importamos los estilos globales.
// Este CSS afecta a toda la aplicacion, por eso se importa en main.jsx.
import './index.css'

// Importamos el componente principal App.
// A partir de App iremos componiendo el resto de la interfaz.
import App from './App.jsx'

// El navegador carga index.html.
// Dentro de index.html existe un div vacio con id="root".
// React usara ese div como contenedor principal de toda la aplicacion.
const contenedor = document.getElementById('root')

// createRoot crea la conexion entre React y el div del HTML.
// render muestra el componente App dentro de ese contenedor.
createRoot(contenedor).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

7.6 `src/App.jsx`

```
// Importamos BrowserRouter desde React Router.
// BrowserRouter activa el sistema de rutas usando la URL del navegador.
// Debe envolver a los componentes que usan rutas o enlaces internos.
import { BrowserRouter } from 'react-router-dom'

// Importamos los estilos propios de App.
// Este archivo se importa aqui porque sus clases se usan en la estructura
general.
import './App.css'

// Importamos la navegacion principal.
// La navegacion se separa en components/ porque no es una pagina completa.
import Navegacion from './components/Navegacion.jsx'

// Importamos la configuracion de rutas.
// AppRoutes vive en routes/ y decide que pagina se muestra segun la URL.
import AppRoutes from './routes/AppRoutes.jsx'

// App es el componente principal de la aplicacion.
// Para clase 2 no metemos aqui el contenido de Inicio ni Contacto.
// App solo monta la estructura comun: router, navegacion y rutas.
function App() {
  return (
    <BrowserRouter>
      <main className="pagina">
        <Navegacion />
        <AppRoutes />
      </main>
    </BrowserRouter>
  )
}

export default App
```

7.7 src/components/Navegacion.jsx

```
// Importamos Link desde React Router.
// Link crea enlaces internos sin recargar toda la pagina.
// Para clase 2 usamos Link porque es la opcion mas sencilla para navegar.
import { Link } from 'react-router-dom'

// Este componente muestra el menu principal de la aplicacion.
// Se coloca en components/ porque es una pieza reutilizable, no una pagina.
// Dejamos los enlaces escritos uno a uno para que el ejemplo sea muy facil de
leer.
function Navegacion() {
  return (
    <nav className="navegacion">
      <Link to="/">
        Inicio
      </Link>

      <Link to="/contacto">
```

```
        Contacto
      </Link>
    </nav>
  )
}

export default Navegacion
```

7.8 src/components/Texto.jsx

```
// Texto es un componente pequeno para separar una parte del contenido.
// Sirve para ver que una pagina puede estar formada por varios componentes.
function Texto() {
  return (
    <p className="texto-apoyo">
      Esta frase viene desde un componente separado llamado Texto.
    </p>
  )
}

// Exportamos Texto para poder usarlo dentro de la pagina Inicio.
export default Texto
```

7.9 src/components/Boton.jsx

```
// Boton es un componente visual muy sencillo.
// De momento no tiene onClick porque los eventos se explicaran mas adelante.
function Boton() {
  return (
    <button className="boton" type="button">
      Ver temario
    </button>
  )
}

// Exportamos Boton para poder usarlo dentro de la pagina Inicio.
export default Boton
```

7.10 src/routes/AppRoutes.jsx

```
// Importamos Routes y Route desde React Router.
// Routes agrupa todas las rutas disponibles.
// Route relaciona una URL con el componente que debe mostrarse.
import { Route, Routes } from 'react-router-dom'

// Importamos las paginas que queremos asociar a una URL.
// Cada pagina vive en su propio archivo para que el proyecto pueda crecer.
import Contacto from '../pages/Contacto.jsx'
import Inicio from '../pages/Inicio.jsx'
```

```
// Este componente centraliza las URLs de la aplicacion.
// Para anadir una pagina nueva, normalmente anadiremos aqui una nueva Route.
// Asi evitamos llenar App.jsx con condicionales o contenidos de todas las
// paginas.
function AppRoutes() {
  return (
    <Routes>
      <Route path="/" element={<Inicio />} />
      <Route path="/contacto" element={<Contacto />} />
    </Routes>
  )
}

export default AppRoutes
```

7.11 src/pages/Inicio.jsx

```
// Importamos componentes pequenos que se muestran dentro de la pagina Inicio.
// Asi vemos que una pagina puede componerse de varias piezas reutilizables.
import Boton from '../components/Boton.jsx'
import Texto from '../components/Texto.jsx'

// En JavaScript usamos const cuando no vamos a reasignar una variable.
// Estos datos se muestran despues dentro del JSX usando llaves: { }.
const tituloCurso = 'Desarrollo de Aplicaciones con React y Node.js'
const duracionHoras = 30
const numeroSesiones = 6
const horasPorSesion = duracionHoras / numeroSesiones

// Inicio es una pagina de la aplicacion.
// Vive en pages/ porque representa una pantalla completa.
// La ruta "/" mostrara este componente gracias a AppRoutes.jsx.
function Inicio() {
  return (
    // Usamos un fragmento <>...</> porque esta pagina devuelve dos secciones.
    // React necesita devolver un unico bloque padre desde cada componente.
    <>
      /* Primer bloque visible de la pagina de inicio. */
      <section className="bloque-presentacion">
        <p className="etiqueta">Dia 1 - Preparacion del entorno</p>

        <h1>{tituloCurso}</h1>

        <p className="entradilla">
          Primer frontend creado con Vite y React para el curso.
        </p>

        <p>
          /* Las llaves permiten insertar valores de JavaScript dentro del
          JSX. */
          El curso tiene {duracionHoras} horas, repartidas en{' '}
          {numeroSesiones} sesiones de {horasPorSesion} horas.
        </p>
      </section>
    </>
  )
}
```



```

    {/* Componentes sencillos incluidos dentro de la pagina Inicio. */}
    <Texto />
    <Boton />
  </section>

  {/* Segundo bloque de la pagina de inicio. */}
  <section className="bloque-conceptos">
    <h2>Que estamos preparando</h2>

    <ul>
      <li>Un proyecto frontend creado con Vite.</li>
      <li>Una aplicacion React muy basica.</li>
      <li>Una base para practicar JavaScript moderno.</li>
    </ul>
  </section>
</>
)
}

export default Inicio

```

7.12 src/pages/Contacto.jsx

```

// Contacto es una pagina de la aplicacion.
// Vive en pages/ porque representa una pantalla completa.
// La ruta "/contacto" mostrara este componente gracias a AppRoutes.jsx.
function Contacto() {
  return (
    // Esta seccion es el contenido completo de la pagina Contacto.
    <section className="bloque-contacto">
      <p className="etiqueta">Pagina interna</p>

      <h1>Contacto</h1>

      <p className="entradilla">
        Esta pagina existe porque la hemos asociado a la URL /contacto.
      </p>

      <p>
        {/* Este texto resume la idea central de la clase 2. */}
        React Router decide que componente debe mostrarse segun la ruta actual.
        Asi podemos anadir nuevas paginas sin mezclar todo dentro de App.jsx.
      </p>
    </section>
  )
}

export default Contacto

```

7.13 src/index.css

```
/* Estilos generales de la pagina.
   Este archivo afecta a toda la aplicacion. */

/* box-sizing hace que los anchos sean mas faciles de calcular.
   Asi padding y border no aumentan el ancho final del elemento. */
* {
  box-sizing: border-box;
}

/* body es la etiqueta general de la pagina HTML.
   Aquí definimos el fondo, el color de texto y la tipografia base. */
body {
  margin: 0;
  background: #f1f5f9;
  color: #0f172a;
  font-family:
    system-ui,
    -apple-system,
    BlinkMacSystemFont,
    'Segoe UI',
    sans-serif;
}

h1,
h2,
p {
  margin-top: 0;
}

/* Estilos base para titulos y parrafos.
   En una app real podrian crecer, pero para clase 2 son suficientes. */
h1 {
  margin-bottom: 16px;
  font-size: 42px;
  line-height: 1.1;
}

h2 {
  margin-bottom: 12px;
  font-size: 24px;
}

p {
  line-height: 1.6;
}
```

7.14 src/App.css

```
/* Contenedor principal de la aplicacion.
   Esta clase se usa en App.jsx.
   Limitamos el ancho para que el contenido sea facil de leer. */
.pagina {
  width: min(900px, calc(100% - 40px));
}
```

```
margin: 0 auto;
padding: 64px 0;
}

/* Menu superior con los enlaces internos.
   Esta clase se usa en Navegacion.jsx. */
.navegacion {
  display: flex;
  gap: 12px;
  margin-bottom: 20px;
}

/* Estilo base de todos los enlaces del menu. */
.navegacion a {
  color: #1e293b;
  font-weight: 700;
  text-decoration: none;
}

/* Estilo cuando el usuario pasa el raton por encima. */
.navegacion a:hover {
  color: #2563eb;
}

/* Bloques blancos que agrupan contenido dentro de cada pagina.
   Estas clases se usan en Inicio.jsx y Contacto.jsx. */
.bloque-presentacion,
.bloque-conceptos,
.bloque-contacto {
  background: #ffffff;
  border: 1px solid #dde3ea;
  border-radius: 8px;
  padding: 28px;
}

/* Separacion entre bloques consecutivos. */
.bloque-conceptos {
  margin-top: 20px;
}

/* Texto pequeno que funciona como etiqueta de contexto. */
.etiqueta {
  margin: 0 0 12px;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
  text-transform: uppercase;
}

/* Parrafo destacado bajo el titulo principal. */
.entradilla {
  color: #334155;
  font-size: 20px;
}

/* Texto que viene del componente Texto.jsx. */
```

```
.texto-apoyo {
  color: #475569;
  font-weight: 700;
}

/* Boton visual del componente Boton.jsx.
   Aun no tiene evento: los eventos se explicaran mas adelante. */
.boton {
  border: 0;
  border-radius: 6px;
  background: #2563eb;
  color: #ffffff;
  cursor: pointer;
  font: inherit;
  font-weight: 700;
  padding: 10px 16px;
}

.boton:hover {
  background: #1d4ed8;
}

/* Ajustes simples para listas dentro del bloque de conceptos. */
.bloque-conceptos ul {
  margin-bottom: 0;
  padding-left: 22px;
}

.bloque-conceptos li + li {
  margin-top: 8px;
}
```

8. Backend preparado

La carpeta `Backend/` forma parte del stack, pero en la clase 2 no desarrollamos Express todavía. Su función es didáctica: mostrar dónde vivirá el servidor cuando lleguemos a Node.

8.1 `Backend/index.js`

```
// Este archivo sera el punto de entrada del backend.  
// Punto de entrada significa: el primer archivo que ejecutaremos con Node.js.  
  
// De momento no arrancamos ningun servidor porque el curso empieza por React.  
// Mas adelante aqui configuraremos Express y las rutas de la API.  
// En clase 2 solo queremos que se vea donde vivira el backend dentro del  
stack.  
  
// React no importara este archivo.  
// Cuando llegue la parte de backend, React se comunicara con este servidor por  
HTTP.  
  
console.log('Backend preparado. Express se configurara mas adelante.')
```

8.2 `Backend/package.json`

```
{  
  "name": "backend",  
  "private": true,  
  "version": "0.0.0",  
  "type": "module",  
  "scripts": {  
    "dev": "node index.js"  
  }  
}
```

Comandos para probarlo:

```
cd Backend  
npm run dev
```

NO HACER

React no debe importar `Backend/index.js`. Cuando el backend exista de verdad, React hablará con él mediante HTTP, por ejemplo con `fetch('/api/contacto')`.

9. JavaScript y JSX usados en este stack

Esta sección conecta el código con la teoría de JavaScript necesaria. No se trata de memorizarlo todo, sino de reconocer las piezas que ya aparecen en el proyecto.

9.1 Imports y exports

Un proyecto React se divide en archivos. Para usar algo definido en otro archivo se importa. Para permitir que otros archivos lo usen, se exporta.

Esta idea es fundamental: cada archivo puede tener una responsabilidad pequeña. Por ejemplo, `Contacto.jsx` solo se preocupa de la página Contacto. Si otro archivo necesita mostrar esa página, la importa. Así evitamos tener todo el proyecto dentro de un único archivo largo.

```
import App from './App.jsx'
export default App
```

Sintaxis	Significado
<code>import App from './App.jsx'</code>	Trae el export por defecto del archivo <code>App.jsx</code> .
<code>import { Link } from 'react-router-dom'</code>	Trae una exportación con nombre llamada <code>Link</code> .
<code>export default Inicio</code>	Permite importar <code>Inicio</code> desde otro archivo.

Ejemplo paso a paso

```
// 1. En Contacto.jsx creamos el componente.
function Contacto() {
  return <h1>Contacto</h1>
}

// 2. Lo exportamos para que otros archivos puedan usarlo.
export default Contacto

// 3. En AppRoutes.jsx lo importamos.
import Contacto from '../pages/Contacto.jsx'
```

Si olvidamos el `export default Contacto`, el archivo `AppRoutes.jsx` no podrá importarlo correctamente. Si escribimos mal la ruta del import, React tampoco encontrará el archivo.

9.2 Constantes

Usamos `const` cuando declaramos un valor que no vamos a reasignar.

```
const tituloCurso = 'Desarrollo de Aplicaciones con React y Node.js'  
const duracionHoras = 30  
const numeroSesiones = 6  
const horasPorSesion = duracionHoras / numeroSesiones
```

Aquí aparecen strings, números y una operación matemática. Después esos valores se insertan en JSX con llaves.

En este ejemplo, `tituloCurso` es una cadena de texto, `duracionHoras` y `numeroSesiones` son números, y `horasPorSesion` es el resultado de una operación. No necesitamos escribir directamente "5"; dejamos que JavaScript lo calcule.

Ejemplo equivalente

```
const duracionHoras = 30  
const numeroSesiones = 6  
  
// JavaScript calcula 30 / 6.  
const horasPorSesion = duracionHoras / numeroSesiones
```

Si más adelante el curso tuviera 40 horas y 8 sesiones, solo habría que cambiar las constantes base. El cálculo seguiría funcionando.

9.3 Funciones como componentes

En React, una forma básica de crear componentes es escribir una función que devuelve JSX.

```
function Contacto() {  
  return (  
    <section>  
      <h1>Contacto</h1>  
    </section>  
  )  
}
```

El nombre del componente se escribe con mayúscula inicial. Por eso usamos `Contacto`, `Inicio` o `Navegacion`.

React distingue entre etiquetas HTML normales y componentes propios. Una etiqueta como `<section>` empieza en minúscula. Un componente como `<Contacto />` empieza en mayúscula. Esta convención ayuda a React a saber qué debe renderizar.

Ejemplo	Qué es
<code><h1></code>	Etiqueta HTML.
<code><section></code>	Etiqueta HTML.

Ejemplo	Qué es
<code><Inicio /></code>	Componente React.
<code><Navegacion /></code>	Componente React.

9.4 JSX

JSX se parece a HTML, pero no es HTML puro. Vive dentro de JavaScript y permite insertar expresiones con llaves.

```
<h1>{tituloCurso}</h1>
<p>El curso tiene {duracionHoras} horas.</p>
```

Todo lo que va entre `{ }` se evalúa como JavaScript. Puede ser una variable, una operación o una expresión sencilla.

Ejemplos de llaves JSX

```
const nombre = 'Ane'
const horas = 30

<p>Hola, {nombre}.</p>
<p>El curso dura {horas} horas.</p>
<p>La mitad del curso son {horas / 2} horas.</p>
```

No usamos llaves para escribir texto fijo. Las usamos cuando queremos que React inserte un valor de JavaScript.

JSX	Resultado visual
<code><p>Contacto</p></code>	Muestra el texto fijo "Contacto".
<code><p>{tituloCurso}</p></code>	Muestra el valor guardado en la constante.

9.5 Componentes dentro de JSX

Un componente puede usar otros componentes como si fueran etiquetas.

```
<Navegacion />
<AppRoutes />
<Texto />
<Boton />
```

La barra final indica que el componente no tiene contenido interno. Es equivalente a abrirlo y cerrarlo sin nada dentro.


```
// Forma autocerrada:
<Navegacion />

// Forma equivalente, pero sin contenido:
<Navegacion></Navegacion>
```

En clase usamos la forma autocerrada porque es más breve y habitual cuando el componente no recibe contenido entre apertura y cierre.

En la página de inicio usamos esta idea para introducir componentes pequeños. `Texto` muestra un párrafo y `Boton` muestra un botón visual. Todavía no tienen props ni eventos porque esta clase se centra en estructura.

```
<Texto />
<Boton />
```

9.6 Props básicas

Las props son valores que pasamos a un componente o etiqueta JSX. En esta clase aparecen props como `to`, `path` y `element`.

```
<Link to="/contacto">Contacto</Link>
<Route path="/contacto" element={<Contacto />} />
```

Prop	Dónde aparece	Qué significa
<code>to</code>	<code>Link</code>	URL interna a la que queremos navegar.
<code>path</code>	<code>Route</code>	URL que React Router debe reconocer.
<code>element</code>	<code>Route</code>	Componente que se renderiza cuando coincide la ruta.
<code>className</code>	Etiquetas JSX	Clase CSS. En JSX se usa <code>className</code> , no <code>class</code> .

Cómo leer una prop

```
<Link to="/contacto">Contacto</Link>
```

Lectura: "crea un enlace interno que navega a `/contacto` y muestra el texto `Contacto`".

```
<Route path="/contacto" element={<Contacto />} />
```

Lectura: "si la URL es `/contacto`, renderiza el componente `Contacto`". La prop `element` usa llaves porque le estamos pasando JSX, no un texto normal.

9.7 Fragmentos

Un componente debe devolver un único bloque padre. Si queremos devolver dos secciones sin crear un div extra, usamos un fragmento.

```
return (  
  <>  
    <section>...</section>  
    <section>...</section>  
  </>  
)
```

9.8 Acceso al DOM

En `main.jsx` aparece una instrucción JavaScript clásica:

```
const contenedor = document.getElementById('root')
```

Significa: busca en el documento HTML el elemento que tenga `id="root"`. Después React monta la aplicación ahí.

Este es uno de los pocos puntos donde tocamos directamente el DOM. A partir de ahí, React se encarga de renderizar la interfaz. Por eso en los componentes no escribimos normalmente `document.getElementById`.

9.9 Llamadas encadenadas

```
createRoot(contenedor).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

Primero se ejecuta `createRoot(contenedor)`. Eso devuelve una raíz de React. Sobre esa raíz llamamos a `.render()` para mostrar `App`.

Podemos leerlo de izquierda a derecha: "crea una raíz de React en este contenedor y renderiza esta aplicación". Aunque parezca una línea compleja al principio, realmente son dos acciones encadenadas.

9.10 Objetos de configuración

En `vite.config.js` aparece un objeto JavaScript:

```
export default defineConfig({  
  plugins: [react()],  
})
```

Las llaves crean un objeto. La propiedad `plugins` guarda un array con el plugin de React.

Objeto y array en el mismo ejemplo

```
{  
  plugins: [react()],  
}
```

El objeto tiene una propiedad llamada `plugins`. El valor de esa propiedad es un array. Dentro del array llamamos a `react()`, que activa el plugin.

RESUMEN JS/JSX

En esta clase no usamos todavía estado, eventos ni formularios. Pero sí usamos conceptos esenciales: módulos, constantes, funciones, JSX, props, fragmentos, objetos y arrays. Es una buena base para pasar después a listas, eventos y estado.

10. Ejercicios de JavaScript y JSX

EJERCICIO JS 1

Constantes y JSX

En `Inicio.jsx`, crea dos nuevas constantes:

```
const ciudad = 'Donostia'  
const modalidad = 'Presencial'
```

Muéstralas en un párrafo usando llaves JSX.

EJERCICIO JS 2

Componente nuevo

Crea un componente llamado `ResumenCurso` dentro de `Inicio.jsx`. Debe devolver un bloque con un título y dos párrafos.

```
function ResumenCurso() {  
  return (  
    <section>  
      <h2>Resumen</h2>  
      <p>Curso de React y Node.js.</p>  
      <p>Duracion: 30 horas.</p>  
    </section>  
  )  
}
```

EJERCICIO JS 3

Props de rutas

Localiza en `AppRoutes.jsx` estas props y explica qué hace cada una:

- `path`
- `element`

EJERCICIO JS 4**Fragmentos**

En `Inicio.jsx`, explica por qué usamos `<>...</>`. Después prueba a quitar el fragmento y observa qué error aparece.

EJERCICIO JS 5**Import y export**

Responde:

1. ¿Qué archivo exporta `Contacto` ?
2. ¿Qué archivo importa `Contacto` ?
3. ¿Qué archivo exporta `Navegacion` ?
4. ¿Qué archivo importa `Navegacion` ?
5. ¿Qué archivo exporta `Texto` ?
6. ¿Qué archivo exporta `Boton` ?
7. ¿Qué archivo importa `Texto` y `Boton` ?

11. Ejercicios de stack

EJERCICIO STACK 1**Reconstruir la arquitectura**

Dibuja en papel o en un archivo de texto la estructura:

```
src/  
  components/  
  pages/  
  routes/  
  Backend/
```

Añade al lado de cada carpeta una frase explicando su función.

EJERCICIO STACK 2**Añadir una página Cursos**

1. Crear `src/pages/Cursos.jsx`.
2. Exportar el componente `Cursos`.
3. Importarlo en `AppRoutes.jsx`.
4. Añadir `<Route path="/cursos" element={<Cursos />} />`.
5. Añadir `<Link to="/cursos">Cursos</Link>` en `Navegacion.jsx`.

EJERCICIO STACK 3**Explicar el flujo de Contacto**

Explica con tus palabras este recorrido:

1. El usuario pulsa `Contacto`.
2. React Router cambia la ruta a `/contacto`.
3. `AppRoutes.jsx` encuentra la ruta.
4. `Contacto.jsx` muestra la página.

EJERCICIO STACK 4**Probar el backend preparado**

Ejecuta:

```
cd Backend
npm run dev
```

Después responde: ¿por qué este backend todavía no se abre en el navegador como una API?

EJERCICIO STACK 5**Preguntas de repaso**

1. ¿Qué archivo contiene `div#root`?

2. ¿Qué archivo monta React dentro de `div#root` ?
3. ¿Qué componente activa las rutas?
4. ¿Qué componente contiene los enlaces?
5. ¿Qué archivo relaciona `/contacto` con `Contacto` ?
6. ¿Qué dependencia se instaló específicamente para las rutas?

Fin de clase 2. La app queda preparada con una estructura mínima, navegación interna y backend reservado. A partir de aquí se puede avanzar hacia props, listas, estado y formularios.